



EAST TEXAS A&M
— UNIVERSITY —

CSCI 380.61E WEB PROGRAMMING AND INTERFACE

INTRODUCTION TO CSS

- CSS stands for Cascading Style Sheets.
- Styles the layout and appearance of web pages.
- Example: Changing colors, fonts, spacing, and positioning of HTML elements.

WHY USE CSS?

- Separation of Concerns: Keeps HTML structure separate from presentation.
- Consistency: Apply the same styles across multiple pages.
- Efficiency: Makes it easier to maintain and update styles.

CSS SYNTAX

- Basic Structure:
 - Selector { property: value; }
- Example:

```
h1 {  
  color: blue;  
  font-size: 24px;  
}
```

CSS SYNTAX

- Components:
 - Selector: h1
 - Property: color, font-size
 - Value: blue, 24px

CSS SELECTORS

- Types of Selectors:
 - Element Selector: h1, p
 - Class Selector: .classname
 - ID Selector: #idname
 - Attribute Selector: [type="text"]
- Example:
`.highlight { background-color: yellow; }`

THE CASCADE AND SPECIFICITY

- Cascade: How styles are applied when multiple rules match.
- Specificity:
 - Inline styles (highest)
 - IDs
 - Classes, attributes
 - Elements (lowest)

THE CASCADE AND SPECIFICITY

- Example:

```
/* This will apply */
```

```
#header {  
  color: red;  
}
```

CSS BOX MODEL

- Components:
 - Content
 - Padding
 - Border
 - Margin
- Visual Explanation: Diagram of the box model.

CSS BOX MODEL

- CSS Example:

```
div {  
    padding: 10px;  
    border: 1px solid black;  
    margin: 20px;  
}
```

LAYOUT TECHNIQUES

- Floats: How to use float for layout.
- Flexbox: Introduction to flexbox layout.
- Grid: Overview of CSS Grid layout.
- Examples: Visual examples of each layout technique.

STYLING TEXT

- Font Properties:
 - Font family, size, weight, style
- Text Properties:
 - Color, alignment, decoration (underline, line-through)
- Example:

```
p {  
  font-family: Arial, sans-serif;  
  color: #333;  
  text-align: center;
```

```
}  
}
```

COLORS AND BACKGROUNDS

- Color Values:
 - Named colors, HEX, RGB, RGBA
- Background Properties:
 - Background-color, background-image, background-size
- Example:

```
body {  
    background-color: #f0f0f0;  
    background-image: url('image.jpg');  
}
```

BORDERS AND SHADOWS

- Border Properties:
 - Width, style, color
- Box Shadow:
 - Horizontal, vertical, blur radius, spread radius, color
- Example:

```
.box {  
  border: 2px solid #000;  
  box-shadow: 5px 5px 10px rgba(0, 0, 0, 0.5);  
}
```

RESPONSIVE DESIGN

- Media Queries: How to apply styles based on device size.
- Example:

```
@media (max-width: 600px) {  
  body {  
    background-color: lightblue;  
  }  
}
```

CSS FRAMEWORKS

- Introduction to CSS Frameworks:
 - Bootstrap, Tailwind, Foundation
- Benefits: Faster development, responsive design, pre-designed components.

CSS PREPROCESSORS

- What are Preprocessors?: Tools like SASS and LESS.
- Benefits: Variables, nesting, mixins, and functions.
- Example:

```
$primary-color: blue;
```

```
.button {  
    background-color: $primary-color;  
}
```

BEST PRACTICES

- Organizing Styles: Using comments, consistent naming conventions.
- Performance: Minifying CSS, using shorthand properties.
- Accessibility: Consider contrast, font sizes, and readability.

COMMON CSS PROPERTIES

- Frequently used properties:
 - Display
 - Overflow
 - Position
 - Float
 - Clear
 - z-index

ADVANCED CSS TECHNIQUES

- Introduction to Advanced CSS
- CSS Grid Layout
- Flexbox Layout
- Responsive Design Techniques
- CSS Variables
- CSS Animations and Transitions
- Best Practices and Tools

INTRODUCTION TO ADVANCED CSS

- **Overview of CSS Capabilities:**
 - Layout, design, responsiveness, and animations.
- **Importance of Mastering Advanced Techniques:**
 - Enhances user experience and accessibility.
- **Modern Browser Support:**
 - Latest versions of Chrome, Firefox, Edge, and Safari.

CSS GRID LAYOUT

- Definition and Purpose:
 - A two-dimensional layout system.
- Key Properties:
 - `grid-template-columns`
 - `grid-template-rows`
 - `grid-area`
- Example Layout:
 - 3-column layout with header and footer.

CSS GRID LAYOUT

```
.container {  
  display: grid;  
  grid-template-columns: 1fr 3fr 1fr;  
  grid-template-rows: auto 1fr auto;  
  grid-template-areas:  
    "header header header"  
    "sidebar content sidebar"  
    "footer footer footer";  
}
```

CSS GRID LAYOUT

```
.header { grid-area: header; }  
.sidebar { grid-area: sidebar; }  
.content { grid-area: content; }  
.footer { grid-area: footer; }
```

- Expected Result:
- A structured layout with designated areas for header, sidebar, content, and footer.

FLEXBOX LAYOUT

- **Introduction to Flexbox:**
 - One-dimensional layout system.
- **Key Properties:**
 - display: flex
 - justify-content
 - align-items
- **When to Use Flexbox vs. Grid:**
 - Use Flexbox for simpler layouts or when items need to align in one dimension.

FLEXBOX EXAMPLE

- Visual Example of a Flex Container:

```
.navbar {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
  background-color: #333;  
  padding: 1rem;  
}
```

FLEXBOX EXAMPLE

- Visual Example of a Flex Container:

```
.nav-item {  
  color: white;  
  text-decoration: none;  
  margin: 0 15px;  
}
```

RESPONSIVE DESIGN TECHNIQUES

- Media Queries:
 - Adjust styles based on device characteristics (width, height, orientation).
- Mobile-first vs. Desktop-first Approaches:
 - Mobile-first: Style for mobile first, then use media queries for larger devices.

MEDIA QUERY EXAMPLE

- Example:

```
body {  
    font-size: 16px;  
}  
  
@media (max-width: 600px) {  
    body {  
        font-size: 14px;  
    }  
}
```

CSS VARIABLES

- Definition and Benefits:
 - Store values in reusable variables for easier maintenance.
- Syntax for Defining and Using Variables:

```
:root {  
  --main-color: #3498db;  
  --padding: 10px;  
}
```

CSS VARIABLES

- Syntax for Defining and Using Variables:

```
.box {  
    background-color: var(--main-color);  
    padding: var(--padding);  
}
```

CSS VARIABLES EXAMPLE

- Code Snippet:

```
<div class="box">This is a box!</div>
```

- Expected Result:
- A box with a background color of #3498db and padding of 10px.

CSS ANIMATIONS AND TRANSITIONS

- Differences between Animations and Transitions:
 - Transitions occur on property changes, while animations are keyframe-based.
- Key Properties:
 - transition
 - @keyframes

CSS ANIMATION EXAMPLE

- Code Example:

```
.button {  
    background-color: #3498db;  
    color: white;  
    padding: 10px 20px;  
    border: none;  
    cursor: pointer;  
    transition: background-color 0.3s;  
}
```

CSS ANIMATION EXAMPLE

- Code Example:

```
.button:hover {  
    background-color: #2980b9;  
}
```

- Expected Result:
 - Button changes color smoothly on hover.

BEST PRACTICES

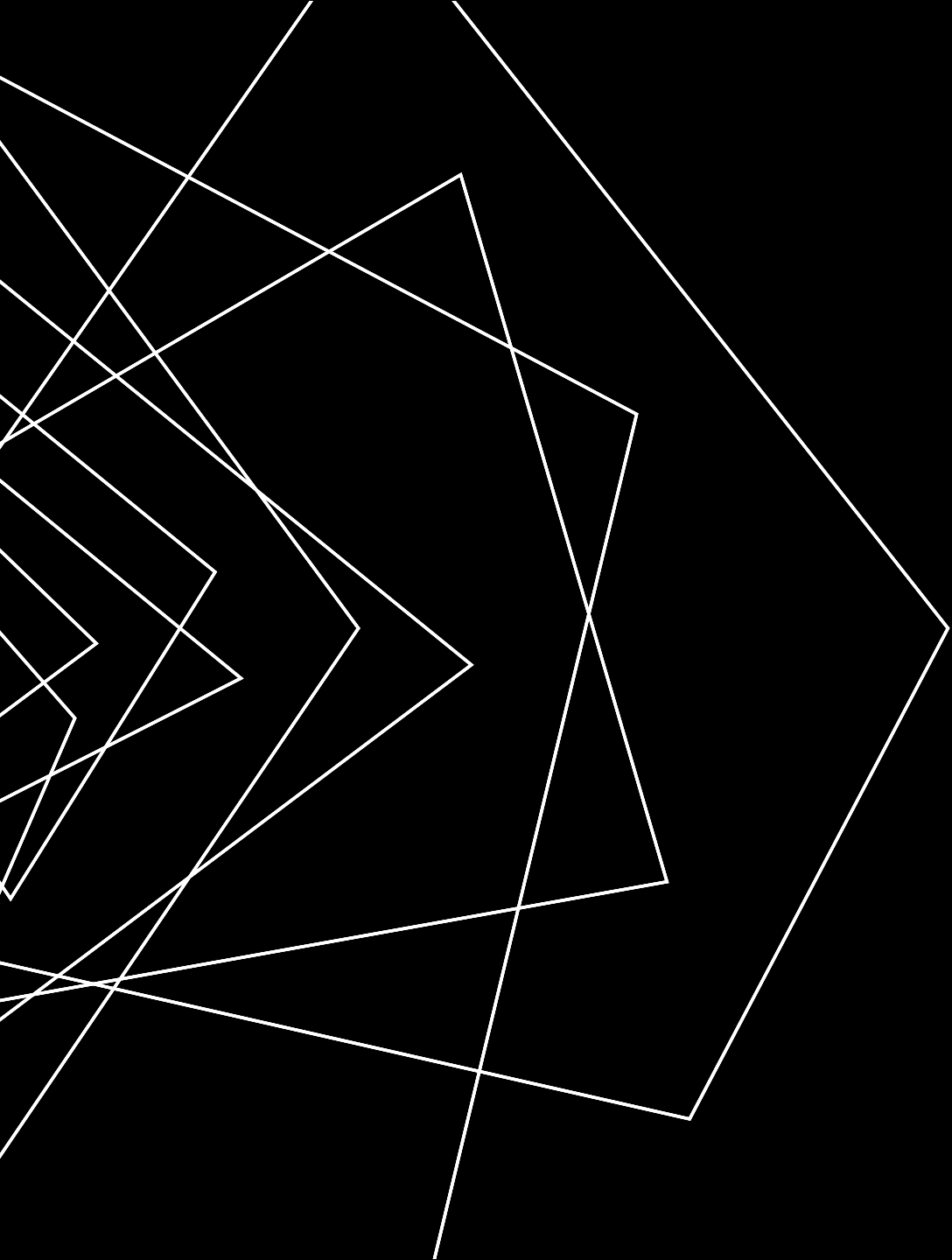
- Organizing CSS:
 - Use methodologies like BEM or SMACSS.
- Performance Considerations:
 - Minimize reflows and repaints.
- Avoiding Specificity Wars:
 - Use class selectors wisely to prevent conflicts.

TOOLS AND RESOURCES

- Recommended Tools:
 - CSS Lint for checking styles.
 - Style guides for consistency.
- Online Resources:
 - MDN Web Docs
 - CSS Tricks
- Browser Developer Tools:
 - Inspect elements, test styles in real-time.

CONCLUSION

- Recap of Topics Covered:
 - CSS Grid, Flexbox, Responsive Design, CSS Variables, Animations.
- Importance of Continual Learning:
 - CSS evolves; staying updated is crucial.
- Encouragement to Experiment:
 - Challenge attendees to apply these techniques in their projects.



THANK YOU

Yau Ka Cheung
Yauka.Cheung@etamu.edu
ACB1 Room 335